

Luniero Marketing Agent — Slash Command Interface

Design Philosophy

Users shouldn't learn a new interface. They already know: - /command syntax from Slack, Discord, Telegram - Natural language descriptions - Conversational follow-ups

Principle: /<action> <natural language description>

Command Categories

1. Content Creation

/write <what you want>

Examples:

```
/write a LinkedIn post about AI trends for 2026
/write a Twitter thread on why startups should use AI automation
/write a blog post about content marketing best practices
/write Instagram captions for our new product launch (5 variations)
/write email sequence for new client onboarding (3 emails)
/write ad copy for Facebook campaign targeting small business owners
```

What happens: 1. Agent parses intent (platform, content type, topic) 2. Loads client context (if client specified or default) 3. Runs: Brief → Draft → Polish → Review pipeline 4. Returns content + asks for approval

Flags:

/write ... --client acme	# Specify client
/write ... --platform linkedin	# Force platform
/write ... --tone casual	# Override tone
/write ... --length short	# Adjust length
/write ... --variations 3	# Multiple versions

2. Quick Content (Skip Full Pipeline)

/quick <what you want>

Examples:

```
/quick LinkedIn post about our new feature
/quick tweet celebrating 1000 followers
/quick Instagram story caption for team photo
```

What happens: - Skips full pipeline - Single LLM call with client context - Faster but less refined - Good for simple, time-sensitive content

3. Content Calendar

/calendar <timeframe> [platform]

Examples:

/calendar this week
/calendar next week linkedin
/calendar february instagram
/calendar q1 all

What happens: 1. Loads client content pillars 2. Researches trending topics 3. Generates content calendar 4. Returns schedule with post ideas

Output:

□ Week of Feb 10-14 – Acme Corp LinkedIn

Mon 10: Thought leadership – "Why async beats meetings"
Tue 11: Product tip – Hidden feature spotlight
Wed 12: Industry news – React to AI announcement
Thu 13: Customer story – Quote from happy client
Fri 14: Weekend thought – Reflection post

Approve this calendar? (yes/edit/regenerate)

4. Reports

/report <type> [timeframe]

Examples:

/report weekly
/report monthly
/report campaign "Summer Launch"
/report performance last 30 days
/report competitor analysis

What happens: 1. Pulls analytics from connected platforms 2. Compiles metrics 3. Generates insights and recommendations 4. Outputs formatted report (text + PDF option)

Output:

□ Acme Corp – Weekly Report (Feb 3-9)

HIGHLIGHTS

□ LinkedIn engagement up 23%
□ Best post: "AI trends" (2.4K impressions)
□ Twitter reach down 8%

TOP CONTENT

1. "Why we ditched spreadsheets" – 847 engagements
2. Product demo video – 412 views
3. Team spotlight – 234 likes

RECOMMENDATIONS

→ Double down on LinkedIn carousels
→ Test Twitter threads instead of links
→ Post customer stories on Wednesdays

Want the full PDF? (/report pdf)

5. Research

/research <topic or request>

Examples:

```
/research trending topics in B2B SaaS
/research what competitors are posting this week
/research best hashtags for project management content
/research viral posts in our industry
/research audience sentiment about AI tools
```

What happens: 1. Research Agent activates 2. Searches web, social platforms, news 3. Compiles findings 4. Returns actionable insights

6. Client Management

/client <action> [details]

Examples:

/client list	# Show all clients
/client switch acme	# Change active client
/client new "Initech Corp" B2B SaaS	# Create new client
/client voice	# Show current client's
brand voice	
/client voice update "more casual"	# Update brand voice
/client pillars	# Show content pillars
/client pillars add "sustainability"	# Add content pillar

Active Client: - One client is "active" at a time - All commands default to active client - Switch with /client switch <name> - Or specify per-command with --client

7. Approvals & Revisions

/approve [job_id]	# Approve pending content
/revise <feedback>	# Request changes
/reject [reason]	# Reject and explain why

Examples:

/approve	# Approve last pending item
/approve job_abc123	# Approve specific job
/revise make it more casual and add a statistic	
/revise stronger hook, remove the question at the end	
/reject off-brand, try again with different angle	

What happens: - /approve → Content moves to next stage or completion - /revise → Sends feedback to Draft/Polish agent, loops back - /reject → Logs feedback, starts fresh

8. Schedule & Publish

```
/schedule <when>          # Schedule approved content
/publish now               # Publish immediately
/queue                    # Show scheduled posts
```

Examples:

```
/schedule tomorrow 9am
/schedule monday 10am EST
/schedule optimal          # Let AI pick best time
/publish now
/queue                    # See what's scheduled
/queue cancel job_abc123  # Cancel scheduled post
```

9. History & Status

```
/status [job_id]          # Check job status
/history [count]          # Recent jobs
/show <job_id>            # Show full output
```

Examples:

```
/status                  # Status of last job
/status job_abc123       # Specific job
/history                 # Last 10 jobs
/history 20              # Last 20 jobs
/show job_abc123        # Full content of a job
```

10. File Uploads

```
/upload <file> [context]
```

Supported Formats: - **Documents:** PDF, DOCX, TXT, MD - **Data:** CSV, XLSX, JSON - **Images:** PNG, JPG, GIF (for visual content analysis) - **Audio:** MP3, WAV (for transcription)

Examples:

```
/upload brand-guidelines.pdf          # Upload brand doc
/upload competitor-report.pdf as research # Upload as research
material
/upload analytics-export.csv          # Upload data
/upload client-brief.docx for next post # Use for specific
job
/upload logo.png                      # Upload image asset
/upload meeting-recording.mp3 transcribe # Transcribe audio
```

What happens:

1. **PDF/DOCX/TXT** → Extracted text, chunked, embedded in vector store
 2. **CSV/XLSX** → Parsed into structured data, available for reports
 3. **Images** → Analyzed with vision model, stored as assets
 4. **Audio** → Transcribed, stored as text
-

Upload to Client Profile:

```
/upload brand-guidelines.pdf --client acme
```

```
/upload voice-examples.pdf --client acme --type voice
/upload competitor-analysis.pdf --client acme --type research
```

Types: - voice — Brand voice/tone examples - guidelines — Brand guidelines - research — Research materials - assets — Images, logos, templates - data — Analytics, reports, spreadsheets - brief — Client briefs, instructions

Use in Content Creation:

```
/write blog post based on uploaded-brief.pdf
/write LinkedIn post using data from q4-analytics.csv
/write Twitter thread summarizing competitor-report.pdf
```

Managing Uploads:

```
/uploads                # List all uploads
/uploads --client acme   # List client uploads
/uploads search "brand guide" # Search uploads
/uploads delete <file_id>  # Delete upload
/uploads show <file_id>    # Preview content
```

Output:

UPLOADS — Acme Corp

ID	NAME	TYPE	SIZE	DATE
file_001	brand-guidelines.pdf	guidelines	2.4MB	Feb 5
file_002	q4-analytics.csv	data	156KB	Feb 6
file_003	competitor-deck.pdf	research	4.1MB	Feb 7
file_004	logo-dark.png	assets	89KB	Feb 7
file_005	meeting-notes.mp3	audio	12MB	Feb 7

5 files · 18.8MB total

Actions:

- /uploads show file_001 Preview file
- /uploads delete file_003 Delete file
- /uploads search "logo" Search files

Bulk Upload:

```
/upload folder ./client-assets/
/upload zip brand-kit.zip
```

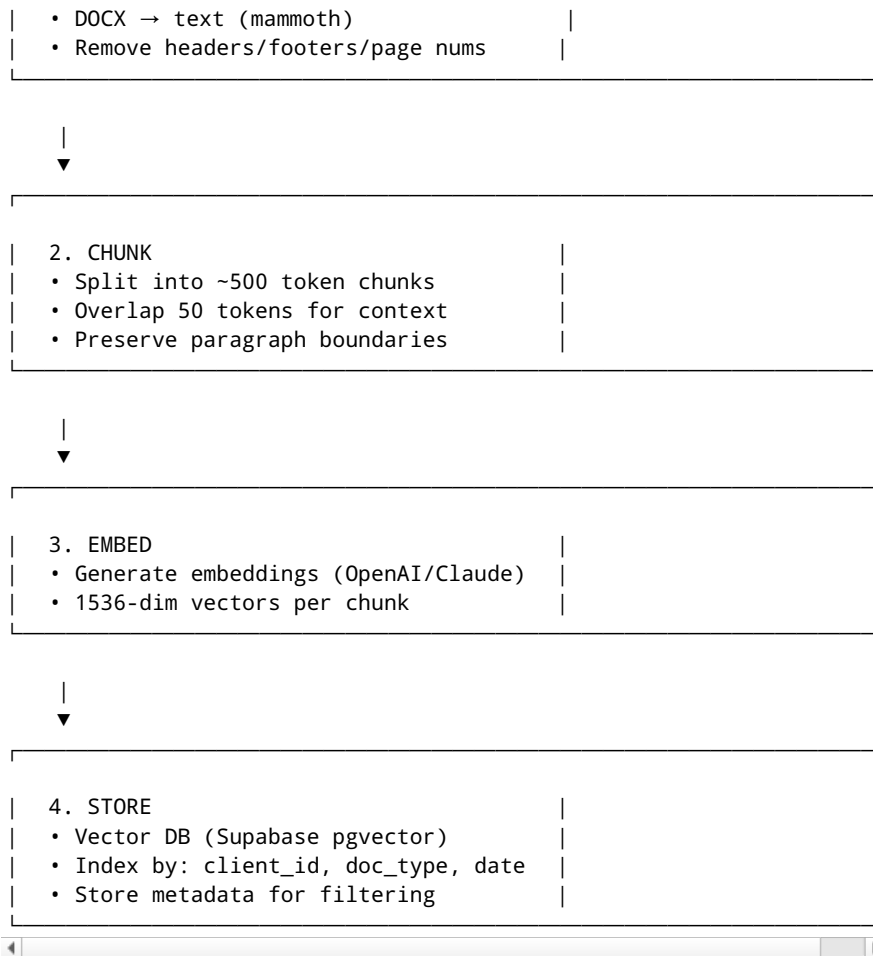
Optimized Storage & Retrieval:

When you upload, the system:

PDF Upload



- | | |
|--------------------------|--|
| 1. EXTRACT | |
| • PDF → text (pdf-parse) | |



Database Schema:

```
CREATE TABLE document_chunks (  
  id UUID PRIMARY KEY,  
  client_id TEXT NOT NULL,  
  document_id UUID NOT NULL,  
  doc_type TEXT, -- 'voice', 'guidelines', 'research', 'brief'  
  chunk_index INTEGER,  
  content TEXT,  
  embedding VECTOR(1536),  
  metadata JSONB,  
  created_at TIMESTAMPTZ  
);  
  
-- Fast retrieval index  
CREATE INDEX ON document_chunks  
  USING ivfflat (embedding vector_cosine_ops)  
  WITH (lists = 100);  
  
-- Filter by client  
CREATE INDEX ON document_chunks (client_id, doc_type);
```

Smart Retrieval:

When writing content, the system automatically:

1. Takes the topic/request
2. Generates query embedding
3. Vector searches relevant chunks (top 5, <200ms)
4. Injects into agent context

/write LinkedIn post about our sustainability efforts

→ Searches uploaded docs for “sustainability” semantically → Finds relevant chunks from brand-guidelines.pdf, esg-report.pdf → Agent uses this context automatically → **User never waits** — retrieval happens in parallel with other setup

Upload in Conversation:

User: [attaches brand-guidelines.pdf]

Agent: Got it! I've processed brand-guidelines.pdf (2.4MB, 47 pages).

Key points extracted:

- Primary tone: Professional but approachable
- Avoid: Jargon, passive voice, exclamation marks
- Keywords: Innovation, efficiency, partnership

Saved to Acme Corp's brand guidelines. Want me to update the brand voice settings?

11. Help & Settings

/help [command]	# Get help
/settings	# Show current settings
/settings <key> <value>	# Update setting

Examples:

/help	# All commands
/help write	# Help for /write
/settings	# Show all settings
/settings default_platform linkedin	
/settings auto_approve false	
/settings timezone America/Chicago	

Conversational Mode

After any command, users can follow up naturally:

User: /write LinkedIn post about AI trends

Agent: Here's your post:
[content]

Approve? (yes/edit/regenerate)

User: make it shorter and punchier

Agent: Here's the revised version:
[shorter content]

User: perfect, schedule it for monday

Agent: Scheduled for Monday Feb 10, 9:00 AM. □

The agent understands context and follow-ups.

Command Aliases

Multiple ways to say the same thing:

Primary	Aliases
/write	/create, /draft, /make
/quick	/q, /fast
/calendar	/cal, /plan
/report	/analytics, /stats
/research	/find, /search
/client	/c
/approve	/ok, /yes, /lgtm
/revise	/edit, /change, /fix
/schedule	/sched, /post
/status	/s, /?
/help	/h, /?

Smart Parsing

The agent intelligently parses natural language:

/write linkedin post about our new feature

Parsed as:

```
{
  "command": "write",
  "platform": "linkedin",
  "content_type": "post",
  "topic": "our new feature",
  "client": "[active_client]"
}
```

/write 5 tweet variations about the product launch with casual tone

Parsed as:

```
{
  "command": "write",
  "platform": "twitter",
  "content_type": "tweet",
  "topic": "product launch",
  "variations": 5,
  "tone": "casual"
}
```

Interactive Prompts

When info is missing, agent asks:

User: /write a blog post

Agent: I'll create a blog post for you.

What topic should I cover?

User: AI automation for small businesses

Agent: Got it. Any specific angle or key points to include?

User: focus on cost savings and time efficiency

Agent: Creating your blog post now...
[runs pipeline]

Progress Animations & Visual Feedback

Users should never see a frozen screen. Every long-running operation shows real-time progress.

Pulsing Circle Animation

A simple, calming animation that shows the system is alive:

- Loading client context...
- ◉ Loading client context...
- Loading client context...
- ◉ Loading client context...
- Loading client context...

The circle pulses (grows and shrinks) continuously — users immediately know the program is working, not frozen.

Implementation:

```
const pulseFrames = ['○', '◉', '●', '◉'];
let frame = 0;

setInterval(() => {
  process.stdout.write(`\r${pulseFrames[frame % 4]} ${message}`);
  frame++;
}, 300);
```

Progress bar (for multi-step operations):

Creating content 40% | Drafting...

Pipeline Progress

When a job runs through the pipeline, show each stage:

✓ Job created: d83c6073-4304-45f3-9d33-9fcddee860d0

- Running pipeline...
 - |— ✓ Context loaded (0.2s)
 - |— ✓ Brief created (2.1s)

```
|— □ Drafting content...
|— ○ Polish
|— ○ Review
```

As each stage completes:

✓ Job created: d83c6073-4304-45f3-9d33-9fcddee860d0

```
□ Running pipeline...
|— ✓ Context loaded (0.2s)
|— ✓ Brief created (2.1s)
|— ✓ Draft created (4.3s)
|— □ Polishing content...
|— ○ Review
```

Final state:

✓ Job created: d83c6073-4304-45f3-9d33-9fcddee860d0

```
✓ Pipeline complete (8.9s)
|— ✓ Context loaded (0.2s)
|— ✓ Brief created (2.1s)
|— ✓ Draft created (4.3s)
|— ✓ Polished (1.8s)
|— ✓ Review passed: 94/100 (0.5s)
```

□ Your content is ready!

Always Explain the Process

Every animation must have explanatory text. Users should always know: 1. What's happening now 2. Why it's happening 3. How long it might take

Format: [Animation] [Action] – [Explanation]

Context stage:

- Loading your brand profile – so the content sounds like you
- ◉ Fetching brand voice – checking your tone preferences
- Pulling content pillars – these guide what topics to focus on
- ◉ Reviewing recent feedback – learning from past content

Brief stage:

- Analyzing your request – understanding what you need
- ◉ Researching angles – finding the best way to approach this
- Structuring the brief – mapping out the content flow
- ◉ Finalizing key messages – picking the most impactful points

Draft stage:

- Writing the first draft – this takes a few seconds
- ◉ Crafting your hook – the first line matters most
- Building the story – connecting your key points
- ◉ Adding details – making it specific and credible
- Wrapping up – ending with a clear call to action

Polish stage:

- Matching your voice – making sure it sounds like your brand
- ◉ Smoothing transitions – improving the flow

- Strengthening the hook – grabbing attention faster
- ⦿ Final polish – small tweaks that make a big difference

Review stage:

- Checking brand fit – does this sound like you?
- ⦿ Validating structure – is the flow logical?
- Scoring engagement – will people want to read this?
- ⦿ Final check – making sure it's ready to publish

Key principle: The user should never wonder “what is it doing?” — the text tells them.

Streaming Output

For content generation, stream the text as it's created:

▯ Drafting content...

The future of AI isn't about replacing humans—it's about amplifying what we do best.

After spending 6 months implementing AI automation for marketing teams, I've seen one pattern emerge again and again: the teams that win aren't the ones who automate everything...■

The text appears character-by-character or chunk-by-chunk, with a cursor showing active generation.

Time Estimates

Show estimated time remaining:

▯ Drafting content... (est. 5s remaining)

Or for longer operations:

▯ Generating monthly report...
Processed 3/12 weeks | ~45s remaining

Error Recovery Animation

If something fails, show clear recovery:

✓ Job created: d83c6073-4304-45f3-9d33-9fcddee860d0

▯ Running pipeline...
 |— ✓ Context loaded (0.2s)
 |— ✓ Brief created (2.1s)
 |— ✓ Draft created (4.3s)
 |— ✓ Polished (1.8s)
 |— ✗ Review failed (timeout)

```
□ Retrying review (attempt 2/3)...
  └─ □ Checking brand alignment...
```

Implementation (src/cli/utlis/spinner.ts)

```
import ora from 'ora';

// Simple spinner
const spinner = ora('Loading...').start();
spinner.succeed('Done!');

// Pipeline progress
class PipelineProgress {
  private stages = ['context', 'brief', 'draft', 'polish',
'review'];
  private current = 0;

  render() {
    const lines = this.stages.map((stage, i) => {
      if (i < this.current) return `└─ ✓ ${stage}`;
      if (i === this.current) return `└─ □ ${stage}...`;
      return `└─ ○ ${stage}`;
    });
    return lines.join('\n');
  }

  advance() {
    this.current++;
    this.redraw();
  }
}

// Rotating messages
const draftMessages = [
  'Writing first draft...',
  'Crafting the hook...',
  'Building the narrative...',
  'Adding supporting points...',
];

let messageIndex = 0;
setInterval(() => {
  spinner.text = draftMessages[messageIndex % draftMessages.length];
  messageIndex++;
}, 2000);
```

Dependency

Add to package.json:

```
{
  "dependencies": {
    "ora": "^6.0.0",
    "cli-progress": "^3.12.0",
    "chalk": "^5.0.0"
  }
}
```

Response Format

All responses follow a consistent format:

Content Output

□ ****LinkedIn Post**** – Acme Corp

[content here]

□ Words: 127 | Score: 94/100

□ #AIautomation #productivity

□ Approve ⇌ Revise □ Regenerate

Status Update

□ Job: job_abc123

□ Status: polishing

□ Client: Acme Corp

□ Type: LinkedIn post

□ Started: 2 min ago

Graceful Issue Handling

Never show scary “ERROR” messages. The task had an issue, not the whole app.

□ Couldn't complete this step

The review took longer than expected. This happens sometimes with complex content.

What now?

→ Retry automatically (recommended)

→ Skip review and show draft

→ Cancel this job

[r] Retry [s] Skip [c] Cancel

For connection issues:

□ Having trouble reaching LinkedIn

This is temporary – their API is responding slowly.
Your content is saved and ready to post when it clears.

→ Try again now

→ Schedule for later

→ Copy content to clipboard

[t] Try [s] Schedule [c] Copy

For validation issues:

□ Quick fix needed

The draft is 47 words over the limit for Twitter.
Want me to trim it, or would you prefer to edit?

→ Auto-trim (keep the best parts)

→ Let me edit it myself

[a] Auto-trim [e] Edit

Key principles: - No “ERROR” or “FAILED” in red - Explain what happened in plain language - Always offer next steps - Keep the user in control - Task failed ≠ app broken

Implementation Architecture

Parser Module (src/cli/parser.ts)

```
interface ParsedCommand {
  command: string;
  subcommand?: string;
  topic?: string;
  platform?: string;
  contentType?: string;
  client?: string;
  flags: Record<string, any>;
  raw: string;
}

function parseCommand(input: string): ParsedCommand {
  // Handle slash commands
  if (input.startsWith('/')) {
    return parseSlashCommand(input);
  }

  // Handle conversational follow-ups
  return parseConversational(input);
}

function parseSlashCommand(input: string): ParsedCommand {
  const [command, ...rest] = input.slice(1).split(' ');
  const content = rest.join(' ');

  // Extract flags (--key value)
  const flags = extractFlags(content);
  const cleanContent = removeFlags(content);

  // Smart parsing based on command
  switch (command) {
    case 'write':
    case 'create':
    case 'draft':
      return parseWriteCommand(cleanContent, flags);
    case 'calendar':
    case 'cal':
      return parseCalendarCommand(cleanContent, flags);
    // ... etc
  }
}
```

```

    }

    function parseWriteCommand(content: string, flags: Record<string,
any>): ParsedCommand {
        // Use NLP or pattern matching to extract:
        // - Platform (linkedin, twitter, instagram, etc.)
        // - Content type (post, thread, carousel, story, etc.)
        // - Topic
        // - Modifiers (casual, short, variations, etc.)

        const platform = extractPlatform(content) || flags.platform;
        const contentType = extractContentType(content) || 'post';
        const topic = extractTopic(content);

        return {
            command: 'write',
            platform,
            contentType,
            topic,
            flags,
            raw: content,
        };
    }
}

```

Command Router (src/cli/router.ts)

```

    async function routeCommand(parsed: ParsedCommand, context:
SessionContext) {
        switch (parsed.command) {
            case 'write':
                return await handleWriteCommand(parsed, context);

            case 'quick':
                return await handleQuickCommand(parsed, context);

            case 'calendar':
                return await handleCalendarCommand(parsed, context);

            case 'report':
                return await handleReportCommand(parsed, context);

            case 'research':
                return await handleResearchCommand(parsed, context);

            case 'client':
                return await handleClientCommand(parsed, context);

            case 'approve':
                return await handleApproveCommand(parsed, context);

            case 'revise':
                return await handleReviseCommand(parsed, context);

            case 'schedule':
                return await handleScheduleCommand(parsed, context);

            case 'status':
                return await handleStatusCommand(parsed, context);

            case 'help':

```

```

        return await handleHelpCommand(parsed, context);

    default:
        // Try to understand as conversational
        return await handleConversational(parsed, context);
    }
}

```

Session Context (src/cli/session.ts)

```

interface SessionContext {
  activeClient: string | null;
  lastJobId: string | null;
  pendingApproval: string | null;
  conversationHistory: Message[];
  settings: UserSettings;
}

class Session {
  private context: SessionContext;

  async processInput(input: string): Promise<Response> {
    // Parse the input
    const parsed = parseCommand(input);

    // Check for conversational context
    if (this.isFollowUp(parsed)) {
      return await this.handleFollowUp(parsed);
    }

    // Route to appropriate handler
    return await routeCommand(parsed, this.context);
  }

  private isFollowUp(parsed: ParsedCommand): boolean {
    // No slash command = likely follow-up
    if (!parsed.raw.startsWith('/')) {
      return this.context.conversationHistory.length > 0;
    }
    return false;
  }

  private async handleFollowUp(parsed: ParsedCommand):
Promise<Response> {
    // Use LLM to understand intent in context
    const intent = await this.classifyIntent(parsed.raw);

    switch (intent) {
      case 'approve':
        return await handleApproveCommand({ command: 'approve' },
this.context);
      case 'revise':
        return await handleReviseCommand({
          command: 'revise',
          topic: parsed.raw
        }, this.context);
      case 'schedule':
        return await handleScheduleCommand({
          command: 'schedule',
          topic: parsed.raw

```

```

    }, this.context);
  default:
    return await handleConversational(parsed, this.context);
  }
}
}

```

Write Command Handler (src/cli/handlers/write.ts)

```

async function handleWriteCommand(
  parsed: ParsedCommand,
  context: SessionContext
): Promise<Response> {
  // Get client
  const clientId = parsed.flags.client || context.activeClient;
  if (!clientId) {
    return {
      type: 'prompt',
      message: 'Which client is this for?',
      options: await listClients(),
    };
  }

  // Create job
  const jobId = await routerAgent.createJob({
    clientId,
    type: mapContentType(parsed.contentType),
    platform: parsed.platform,
    topic: parsed.topic,
    instructions: parsed.flags.instructions,
  });

  // Update session
  context.lastJobId = jobId;
  context.pendingApproval = jobId;

  // Wait for completion (with streaming updates)
  return await streamJobProgress(jobId);
}

async function streamJobProgress(jobId: string): Promise<Response> {
  return new Promise((resolve) => {
    const checkInterval = setInterval(async () => {
      const job = await stateStore.getJob(jobId);

      // Send progress update
      emitProgress({
        jobId,
        status: job.status,
        stage: getStageLabel(job.status),
      });

      // Check if complete
      if (job.status === 'complete' || job.status ===
'human_review') {
        clearInterval(checkInterval);
        resolve(formatContentResponse(job));
      }

      if (job.status === 'failed') {

```

```
        clearInterval(checkInterval);
        resolve(formatErrorResponse(job));
    }
    }, 1000);
});
}
```

Complete Command Reference

Content Commands

Command	Description	Example
/write <description>	Create content with full pipeline	/write LinkedIn post about AI
/quick <description>	Fast single-shot content	/quick tweet about our sale
/calendar <timeframe>	Generate content calendar	/calendar next week
/repurpose <source> to <target>	Convert content	/repurpose blog to twitter thread

File Commands

Command	Description	Example
/upload <file>	Upload file for processing	/upload brand- guide.pdf
/uploads	List uploaded files	/uploads --client acme
/uploads show <id>	Preview file content	/uploads show file_001
/uploads delete <id>	Delete uploaded file	/uploads delete file_003

Analytics Commands

Command	Description	Example
/report <type>	Generate report	/report monthly
/research <topic>	Research a topic	/research competitor content
/trending	Get trending topics	/trending in B2B SaaS

Client Commands

Command	Description	Example
/client list	List all clients	/client list
/client switch <name>	Change active client	/client switch acme

/client new <name> <industry>	Create client	/client new "Acme" SaaS
/client voice	View brand voice	/client voice
/client pillars	View content pillars	/client pillars

Workflow Commands

Command	Description	Example
/approve	Approve pending content	/approve
/revise <feedback>	Request changes	/revise make it shorter
/reject <reason>	Reject content	/reject off-brand
/schedule <when>	Schedule content	/schedule monday 9am
/publish	Publish immediately	/publish now

Utility Commands

Command	Description	Example
/status	Check job status	/status
/history	View recent jobs	/history 10
/queue	View scheduled posts	/queue
/help	Get help	/help write
/settings	View/update settings	/settings timezone EST

Debugging & Troubleshooting

11. Debug Commands

/debug <action> [options]

Core Debug Commands:

/debug status	# System health overview
/debug job <job_id>	# Deep inspect a job
/debug agents	# Agent status and health
/debug logs [count]	# Recent system logs
/debug trace <job_id>	# Full event trace for a job
/debug config	# Show current configuration
/debug connections	# Test all API connections

System Status

/debug status

Output:

▣ SYSTEM STATUS

- ▣ Redis Connected (latency: 2ms)
- ▣ Supabase Connected (latency: 45ms)

```
□ Claude API      Active (quota: 89% remaining)
□ Twitter API     Rate limited (resets in 12m)
□ LinkedIn API    Connected
```

AGENTS

```
├─ □ Router      Running (jobs: 47 today)
├─ □ Context     Running (avg: 120ms)
├─ □ Brief       Running (avg: 2.1s)
├─ □ Draft       Running (avg: 4.3s)
├─ □ Polish      Running (avg: 1.8s)
└─ □ Review      Running (avg: 1.2s)
```

QUEUES

```
├─ Pending jobs: 3
├─ Processing:  1
├─ Failed (24h): 2
└─ Completed (24h): 44
```

Last error: 2h ago (rate limit on Twitter)

Job Inspection

/debug job <job_id>

Output:

□ JOB INSPECTION: job_abc123

METADATA

```
├─ ID:           job_abc123
├─ Client:       acme
├─ Type:         linkedin_post
├─ Status:       failed
├─ Created:      2026-02-07 14:23:01 UTC
├─ Updated:      2026-02-07 14:23:45 UTC
├─ Duration:     44s
└─ Iterations:   2/3
```

INPUT

```
{
  "topic": "AI automation benefits",
  "platform": "linkedin",
  "instructions": "include statistics"
}
```

PIPELINE STATUS

```
├─ □ context_loaded  (took 0.12s)
├─ □ brief_created   (took 2.3s)
├─ □ draft_created   (took 4.1s)
├─ □ polish_done     (took 1.9s)
├─ □ review          (FAILED)
└─ □ complete        (not reached)
```

ERROR

```
{
  "stage": "review",
  "error": "API timeout after 30s",
  "stack": "TimeoutError at ReviewAgent.callLLM..."
}
```

```
}
```

ACTIONS

- /debug retry job_abc123 Retry from failed stage
- /debug trace job_abc123 See full event trace
- /debug dump job_abc123 Export full job data

Event Trace

/debug trace <job_id>

Output:

□ EVENT TRACE: job_abc123

```
14:23:01.123 job.created
             └─ source: router
             └─ payload: {type: "linkedin_post", topic: "AI
automation"}

14:23:01.245 context.loaded
             └─ source: context-agent
             └─ duration: 122ms
             └─ payload: {client: "acme", pillars: 4, voice:
"professional"}

14:23:03.567 brief.ready
             └─ source: brief-agent
             └─ duration: 2322ms
             └─ payload: {wordCount: 150, sections: 3}

14:23:07.891 draft.ready
             └─ source: draft-agent
             └─ duration: 4324ms
             └─ payload: {wordCount: 147, iteration: 1}

14:23:09.234 polish.done
             └─ source: polish-agent
             └─ duration: 1343ms

14:23:09.456 → review started
             └─ source: review-agent

14:23:39.456 □ agent.error
             └─ source: review-agent
             └─ error: "TimeoutError: API call exceeded 30s"
             └─ stack: [expandable]
```

```
Total duration: 38.3s
Events: 7
Errors: 1
```

Agent Health

/debug agents

Output:

▢ AGENT HEALTH

ROUTER

└─ Status: ▢ Running
└─ Jobs today: 47
└─ Avg latency: 45ms
└─ Errors (24h): 0

CONTEXT-AGENT

└─ Status: ▢ Running
└─ Calls today: 47
└─ Avg latency: 120ms
└─ Cache hits: 89%
└─ Errors (24h): 0

BRIEF-AGENT

└─ Status: ▢ Running
└─ Calls today: 47
└─ Avg latency: 2.1s
└─ Tokens used: 45,230
└─ Errors (24h): 1

DRAFT-AGENT

└─ Status: ▢ Running
└─ Calls today: 52 (includes revisions)
└─ Avg latency: 4.3s
└─ Tokens used: 124,500
└─ Errors (24h): 0

POLISH-AGENT

└─ Status: ▢ Running
└─ Calls today: 48
└─ Avg latency: 1.8s
└─ Tokens used: 38,900
└─ Errors (24h): 0

REVIEW-AGENT

└─ Status: ▢ Degraded (1 timeout)
└─ Calls today: 52
└─ Avg latency: 1.2s
└─ Pass rate: 87%
└─ Tokens used: 28,400
└─ Errors (24h): 2

ACTIONS

- /debug agent restart <name> Restart an agent
 - /debug agent logs <name> View agent-specific logs
-

Logs

/debug logs [count] [--level <level>] [--agent <name>]

Examples:

```
/debug logs                           # Last 20 logs  
/debug logs 50                        # Last 50 logs  
/debug logs --level error             # Only errors  
/debug logs --agent draft-agent       # Only from draft agent
```

```
/debug logs 100 --level warn      # Last 100 warnings
```

Output:

```
▢ SYSTEM LOGS (last 20)
```

```
14:45:23 INFO [router] Job created: job_def456
14:45:23 DEBUG [context-agent] Loading client: acme
14:45:23 DEBUG [context-agent] Cache hit for brand voice
14:45:23 INFO [context-agent] Context loaded in 89ms
14:45:25 INFO [brief-agent] Brief created: 145 words
14:45:29 INFO [draft-agent] Draft created: 142 words
14:45:31 INFO [polish-agent] Polish complete
14:45:32 INFO [review-agent] Review passed: 92/100
14:45:32 INFO [router] Job complete: job_def456

14:23:39 ERROR [review-agent] TimeoutError: API call exceeded 30s
└─ job_id: job_abc123
└─ iteration: 2
└─ stack: TimeoutError at ReviewAgent.callLLM
          at processEvent (review-agent.ts:45)
          at MessageBus.consume (message-bus.ts:89)
```

```
[... more logs ...]
```

```
Showing 20 of 1,247 logs today
```

```
Filter: /debug logs --level <debug|info|warn|error>
```

Connection Testing

```
/debug connections
```

Output:

```
▢ CONNECTION TEST
```

```
Testing Redis...
```

```
└─ Host:      localhost:6379
└─ Ping:      ▢ 2ms
└─ Streams:   ▢ Active (3 consumers)
└─ Memory:    45MB / 256MB
```

```
Testing Supabase...
```

```
└─ Host:      xxx.supabase.co
└─ Ping:      ▢ 45ms
└─ Auth:      ▢ Valid
└─ Tables:    ▢ Accessible (jobs, clients, feedback)
└─ Storage:   ▢ Connected
```

```
Testing Claude API...
```

```
└─ Endpoint:  api.anthropic.com
└─ Ping:      ▢ 120ms
└─ Auth:      ▢ Valid (sk-ant...XXX)
└─ Quota:     89% remaining
└─ Model:     claude-sonnet-4-20250514 ✓
```

```
Testing Twitter API...
```

```
└─ Endpoint:  api.twitter.com
```

|— Auth: ✅ Valid
|— Rate: ✅ Limited (resets 14:55)
└─ Quota: 12/300 remaining

Testing LinkedIn API...

|— Endpoint: api.linkedin.com
|— Auth: ✅ Valid
└─ Quota: 450/500 remaining

All critical connections OK ✅
1 warning: Twitter rate limited

Configuration

/debug config

Output:

✅ CONFIGURATION

ENVIRONMENT

|— Node env: production
|— Version: 1.0.0
|— Started: 2026-02-07 08:00:00 UTC
└─ Uptime: 6h 45m

AGENTS

|— Model: claude-sonnet-4-20250514
|— Max tokens: 4096
|— Temperature: 0.7 (draft), 0.5 (polish), 0.3 (review)
└─ Timeout: 30s

LIMITS

|— Max iterations: 3
|— Max concurrent: 10
|— Rate limit: 100 jobs/hour
└─ Max job age: 24h

CLIENTS

|— Default client: acme
|— Active clients: 5
└─ Total clients: 12

STORAGE

|— Redis URL: localhost:6379
|— Supabase project: xxx
└─ Local memory: /app/memory/clients/

SENSITIVE VALUES HIDDEN

Use /debug config --show-secrets to reveal (admin only)

Recovery Actions

/debug retry <job_id> [--from <stage>] # Retry a failed job
/debug cancel <job_id> # Cancel a stuck job
/debug reset <job_id> # Reset job to initial
state

```
/debug flush queue           # Clear pending jobs
(debug!)
/debug agent restart <name>   # Restart specific agent
/debug agent restart all      # Restart all agents
```

Examples:

```
/debug retry job_abc123      # Retry from failed
stage
/debug retry job_abc123 --from draft # Retry from draft stage
/debug cancel job_xyz789     # Cancel stuck job
/debug agent restart review-agent # Restart review agent
```

Output:

```
❏ RETRYING JOB: job_abc123
```

```
Original failure: review (TimeoutError)
Retrying from: review stage
Previous iterations: 2
New iteration: 3
```

```
❏ Sending to review-agent...
❏ Review passed: 88/100
❏ Job complete!
```

```
Job recovered successfully.
Output: /show job_abc123
```

Data Export

```
/debug dump <job_id> [--format <json|yaml>] # Export job data
/debug dump logs [--since <time>]           # Export logs
/debug dump client <client_id>              # Export client data
```

Examples:

```
/debug dump job_abc123          # Export as JSON
/debug dump job_abc123 --format yaml # Export as YAML
/debug dump logs --since "1 hour ago" # Export recent logs
/debug dump client acme         # Export client
profile
```

Interactive Debug Mode

```
/debug shell
```

Enters an interactive debugging shell:

```
❏ DEBUG SHELL (type 'exit' to leave)
```

```
debug> status
[shows system status]
```

```
debug> job job_abc123
[shows job details]
```

```
debug> redis KEYS agent:*
1) "agent:router:stats"
2) "agent:brief:stats"
...

debug> sql SELECT * FROM jobs WHERE status = 'failed' LIMIT 5
[runs Supabase query]

debug> eval stateStore.getJob('job_abc123')
[executes code in context]

debug> restart review-agent
Restarting review-agent... done.

debug> exit
Exiting debug shell.
```

Debug Command Reference

Command	Description
/debug status	System health overview
/debug job <id>	Inspect specific job
/debug trace <id>	Full event trace
/debug agents	All agent health
/debug agent <name>	Specific agent details
/debug logs [n]	Recent logs
/debug config	Show configuration
/debug connections	Test all connections
/debug retry <id>	Retry failed job
/debug cancel <id>	Cancel stuck job
/debug reset <id>	Reset job state
/debug agent restart <name>	Restart agent
/debug dump <target>	Export data
/debug shell	Interactive debug mode

Debug Flags (Global)

Add to any command for extra debugging info:

```
--verbose      # Extra output
--timing        # Show timing breakdown
--raw          # Show raw API responses
--dry-run      # Simulate without executing
```

Examples:

```
/write LinkedIn post about AI --verbose --timing
/debug job job_abc123 --raw
/schedule monday 9am --dry-run
```

Error Handling

Missing Information

User: /write a post

Agent: I'd love to help! A few quick questions:

1. What platform? (LinkedIn, Twitter, Instagram, etc.)
2. What topic should the post cover?

Or just tell me naturally: "LinkedIn post about our new feature"

Invalid Command

User: /postify trending content

Agent: I don't recognize `/postify`. Did you mean:

- `/write` – Create content
- `/quick` – Quick content
- `/research` – Research topics

Type `/help` for all commands.

Processing Error

User: /report monthly

Agent: ☐ Couldn't generate the report.

Issue: LinkedIn API returned an error (rate limited)

Options:

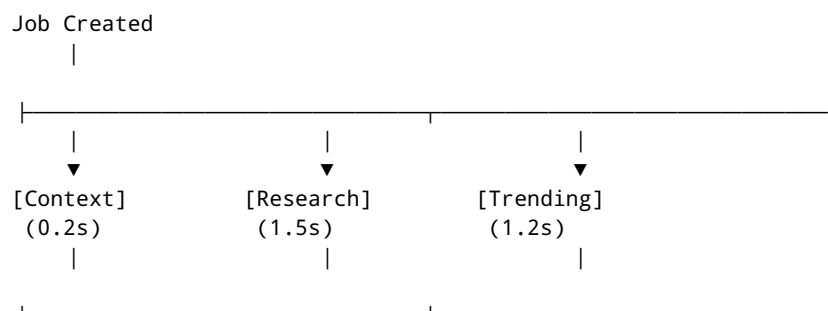
1. Try again in 15 minutes
 2. Generate report without LinkedIn data: `/report monthly --skip linkedin`
 3. Check API status: `/status apis`
-

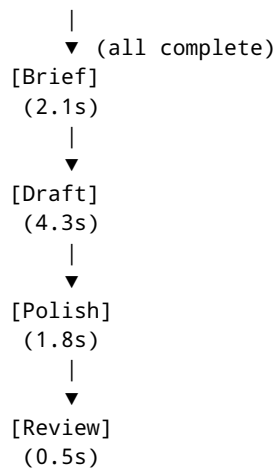
Parallel vs Linear Processing

Problem: Complex tasks timeout when everything runs sequentially.

Solution: Run independent tasks in parallel, dependent tasks in sequence.

What Runs in Parallel





Parallel (independent): - Context loading - Research/trending topics - Asset fetching - Analytics pulling

Sequential (dependent): - Brief → Draft → Polish → Review - Each needs the previous output

Speed Comparison

Approach	Total Time
All sequential	11.6s
With parallel	8.9s (23% faster)

Implementation

```
async function runPipeline(job: Job) {
  // Phase 1: Parallel (independent tasks)
  const [context, research, trending] = await Promise.all([
    contextAgent.load(job.clientId),
    researchAgent.analyze(job.topic),
    trendingAgent.fetch(job.industry),
  ]);

  // Phase 2: Sequential (dependent tasks)
  const brief = await briefAgent.create({ context, research,
trending, job });
  const draft = await draftAgent.write(brief);
  const polished = await polishAgent.refine(draft);
  const review = await reviewAgent.check(polished);

  return { polished, review };
}
```

Timeout Handling

Don't let one slow task kill the whole job:

```
// Wrap parallel tasks with individual timeouts
const [context, research, trending] = await Promise.all([
  withTimeout(contextAgent.load(job.clientId), 5000, 'context'),
  withTimeout(researchAgent.analyze(job.topic), 10000, 'research'),
  withTimeout(trendingAgent.fetch(job.industry), 10000, 'trending'),
]);
```

```

]);

// If research times out, continue without it
function withTimeout(promise, ms, name) {
  return Promise.race([
    promise,
    new Promise((_, reject) =>
      setTimeout(() => reject({ timeout: true, task: name }), ms)
    )
  ]).catch(err => {
    if (err.timeout) {
      console.log(`• ${name} took too long, continuing without it`);
      return null; // Return null, don't fail the whole job
    }
    throw err;
  });
}

```

User Feedback During Parallel

Show what's happening simultaneously:

- Running pipeline...

In parallel:

- | | |
|------------------------|---------------|
| • Loading context... | ✓ done (0.2s) |
| • Researching topic... | • working... |
| • Fetching trends... | ✓ done (1.1s) |

Waiting for research to complete...

Then transition to sequential:

✓ Parallel tasks complete (1.5s)

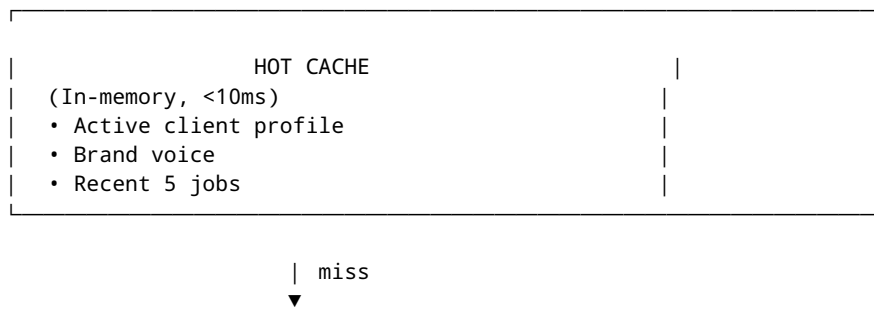
Now creating your content:

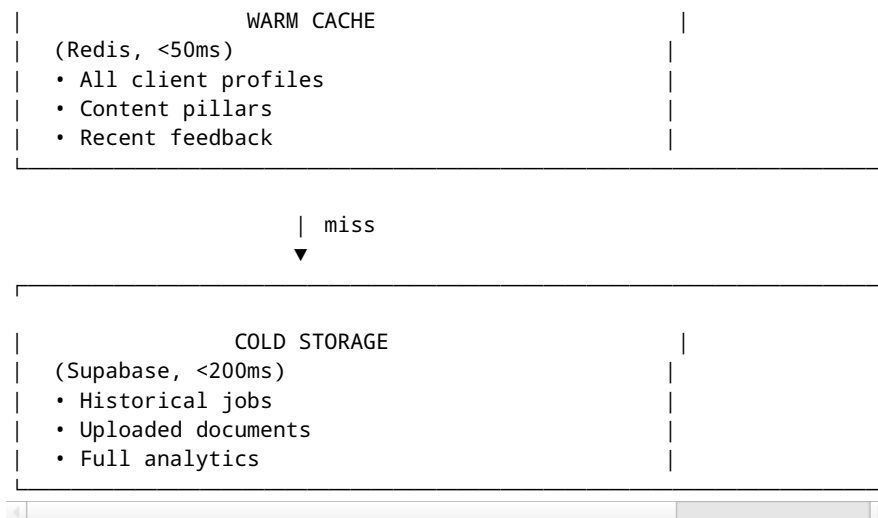
- Writing brief...

Fast Memory Retrieval

Goal: Memory lookups under 500ms. Users shouldn't wait for context.

Caching Strategy





Pre-loading

When a client is selected, preload their data:

```
async function switchClient(clientId: string) {  
  // Immediately cache everything we'll need  
  await Promise.all([  
    cache.set(`profile:${clientId}`, await db.getProfile(clientId)),  
    cache.set(`voice:${clientId}`, await  
db.getBrandVoice(clientId)),  
    cache.set(`pillars:${clientId}`, await  
db.getContentPillars(clientId)),  
    cache.set(`feedback:${clientId}`, await  
db.getRecentFeedback(clientId, 10)),  
  ]);  
  
  activeClient = clientId;  
  console.log(`✓ Switched to ${clientId} (context preloaded)`);  
}
```

Lazy Loading for Documents

Don't load full PDFs into memory. Use vector search:

```
async function getRelevantContext(topic: string, clientId: string) {  
  // Vector search returns only relevant chunks (<500ms)  
  const chunks = await vectorStore.search({  
    query: topic,  
    filter: { client_id: clientId },  
    limit: 5,  
    threshold: 0.7,  
  });  
  
  return chunks.map(c => c.content).join('\n\n');  
}
```

Timing Targets

Operation	Target	Max
Profile lookup		

	<10ms	50ms
Brand voice	<10ms	50ms
Vector search	<200ms	500ms
Full context load	<300ms	1000ms

If any operation exceeds max, log a warning and continue with partial data.

File Structure for CLI Module

```
src/cli/
├── index.ts           # Main entry point
├── parser.ts         # Command parsing
├── router.ts         # Command routing
├── session.ts        # Session management
├── formatter.ts      # Response formatting
├── handlers/
│   ├── write.ts      # /write command
│   ├── quick.ts      # /quick command
│   ├── calendar.ts   # /calendar command
│   ├── report.ts     # /report command
│   ├── research.ts   # /research command
│   ├── client.ts     # /client commands
│   ├── approval.ts   # /approve, /revise, /reject
│   ├── schedule.ts   # /schedule, /publish, /queue
│   ├── status.ts     # /status, /history
│   └── help.ts       # /help, /settings
└── utils/
    ├── nlp.ts        # Natural language parsing
    ├── streaming.ts   # Progress streaming
    └── prompts.ts     # Interactive prompts
```

Copy this entire document into Claude Code and say: "Add this CLI interface to the Luniero implementation. Update the project structure and implement the parser, router, and handlers."